

บทที่ 3

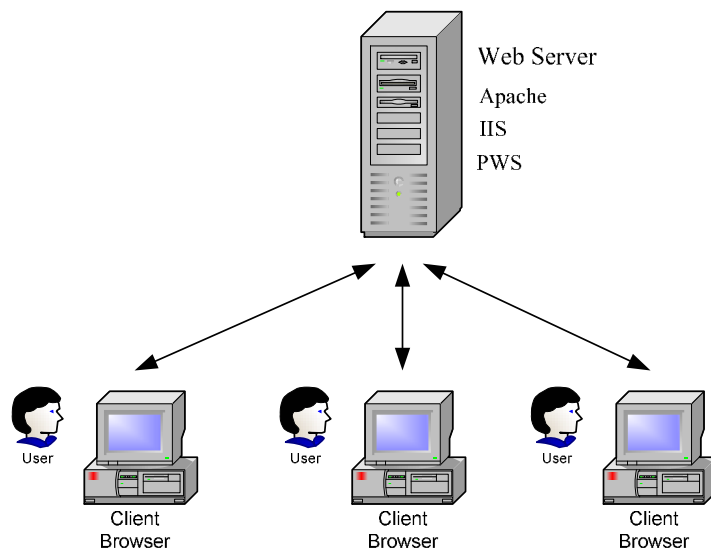
ทฤษฎีพื้นฐานทางอินเทอร์เน็ต

3.1 โครงสร้างพื้นฐานของเวิลด์ไวด์เว็บ (World Wide Web-www)

เวิลด์ไวด์เว็บ (WWW) แปลความหมายเป็นภาษาไทยอย่างตรงๆ แปลความได้ว่า สายใย กว้างไกล ครอบคลุมทั่วโลก หรือเครือข่ายใยแมงมุม

เวิลด์ไวด์เว็บ (WWW) เป็นการบริการข้อมูลข่าวสารที่มีเขตครอบคลุมทั่วทั้งโลก ในแบบสื่อผสมหรือมัลติมีเดีย (Multimedia) กล่าวคือ ข้อมูลเหล่านี้จะประกอบไปด้วยข้อความ ภาพ และเสียงประกอบกัน จึงเป็นที่นิยม ของคนทั่วโลก ข้อมูลนี้จะถูกแบ่งเป็นหน้าๆ โดยแต่ละหน้าจะถูกเขียนขึ้นโดยภาษาทางคอมพิวเตอร์ ซึ่งก็คือ ภาษาเอชทีเอ็มแอล (HTML -HyperText Markup Language) นั่นเอง ข้อมูลต่างๆ สามารถเชื่อมโยงถึงกันได้ โดยไม่จำเป็นต้องอยู่ทีเดียวกัน

เวิลด์ไวด์เว็บ (WWW) เปรียบเสมือนห้องสมุดใหญ่ๆ แห่งหนึ่ง ที่มีหนังสือครบทุกประเภท ตั้งแต่ เทพนิยายจนถึงผลงานวิจัยทาง วิทยาศาสตร์ เราสามารถเข้าไปอ่านได้โดยไม่ต้องลุกไปไหนเลย เพียงแต่เรารู้ที่อยู่ของหนังสือเล่มนั้นก็พอ



รูปที่ 3.1 ส่วนประกอบพื้นฐานของเวิลด์ไวด์เว็บ www

เว็บ คือ รูปแบบของข้อมูลที่ใช้ในการติดต่อสื่อสารบนเครือข่ายอินเทอร์เน็ต ซึ่งเกี่ยวข้องกับหลายๆ ส่วน มีส่วนที่สำคัญ ดังนี้

เวิลด์ไวด์เว็บเซิร์ฟเวอร์ (WWW Server)

เวิลด์ไวด์เว็บเซิร์ฟเวอร์ เป็นแหล่งข้อมูลในระบบ หมายถึง เครื่องคอมพิวเตอร์หรือศูนย์คอมพิวเตอร์ในเครือข่ายอินเทอร์เน็ต ซึ่งเซิร์ฟเวอร์จะให้บริการข้อมูลชนิดที่เรียกว่า ข้อมูลเอชทีเอ็มแอล(HTML)

เอชทีเอ็มแอล (HTML)

เอชทีเอ็มแอล (HTML) เป็นภาษาสำหรับเขียนไฟล์ข้อมูลแบบไฮเปอร์เท็กซ์ ซึ่งเป็นไฟล์ข้อมูลที่ใช้ในระบบเวิลด์ไวด์เว็บ หรือไฟล์แสดงโฮมเพจ ดังนั้นจึงเรียกไฟล์ข้อมูลชนิดนี้ว่า ไฟล์ข้อมูลเอชทีเอ็มแอล ไฟล์ชนิดนี้ประกอบด้วยข้อมูลประเภทต่างๆ ได้แก่ ข้อความ รูปภาพ เสียง และวิดีโอ เป็นต้น ไฟล์เอชทีเอ็มแอลที่แสดงผ่านโปรแกรมบราวเซอร์เช่นนี้ถูกเรียกว่า โฮมเพจ

ไฮเปอร์ลิงก์ (Hyperlink)

ไฮเปอร์ลิงก์ (Hyperlink) เป็นคำหรือวลีของเว็บเพจ หรือโฮมเพจ ซึ่งเกิดจากไฟล์ข้อมูล เอชทีเอ็มแอล ไฮเปอร์เท็กซ์ เป็นคำหรือวลีพิเศษที่เป็นจุดเชื่อมโยงแหล่งข้อมูลได้โดยใช้เมาส์คลิก (click) ไปยังคำหรือวลีนั้น

เว็บไซต์ (Web Site)

เว็บไซต์เป็นเครื่องมือที่ใช้ในการจัดเก็บเว็บเพจของแต่ละองค์กร ที่จะนำเสนอข้อมูลของตนในรูปแบบของเว็บ องค์กรต่างๆ มักจะมีเว็บไซต์เป็นของตนเอง และมักใช้ชื่อองค์กรเป็นชื่อเว็บไซต์ เพื่อให้สามารถจดจำได้ง่าย นอกจากนี้ยังมีกลุ่มคนที่ต้องการเผยแพร่ความรู้ เทคนิคต่างๆ จึงได้ตั้งเว็บไซต์ของตนเองขึ้นมาด้วย

โฮมเพจ (Home Page)

โฮมเพจ คือ เว็บเพจหน้าแรกของข้อมูลแต่ละเรื่อง ซึ่งเปรียบเสมือนหน้าปกของหนังสือแต่ละเล่ม ในส่วนของ โฮมเพจนี้จะเป็นส่วนที่บอกให้ทราบว่าข้อมูลนี้เป็นเรื่องใด พร้อมกับมีสารบัญในการเลือกไปยังหัวข้อต่างๆ

เว็บเพจ (Web Page)

เว็บเพจ คือ เอกสารข้อมูลในแต่ละหน้า ซึ่งจะประกอบไปด้วยข้อความ รูป ภาพเคลื่อนไหว และเสียง จึงเป็นข้อมูลแบบสื่อผสมหรือมัลติมีเดีย

เว็บเบราว์เซอร์ (Web Browser)

เป็นโปรแกรมที่ทำหน้าที่อ่านและแสดงผลเว็บเพจซึ่งเป็นเอกสารข้อมูลที่เขียนขึ้นโดยภาษาเอชทีเอ็มแอล (HTML) เช่น โปรแกรมอินเทอร์เน็ตเอกซ์พลอเรอร์ (Internet Explorer), Netscape, นีโอพลาเน็ต (NeoPlanet), โอเปร่า (Opera) และไทยเบราว์เซอร์ (ThaiBrowser)

โปรโตคอลเอชทีทีพี (HTTP-Protocol HTTP)

โปรโตคอลเอชทีทีพี (HTTP) นี้สร้างขึ้นสำหรับการบริการเวิลด์ไวด์เว็บ (WWW) ในเครือข่ายอินเทอร์เน็ตโดยเฉพาะ โปรโตคอลนี้จะเป็นตัวกำหนดวิธีการส่งข้อมูลหรือไฟล์ระหว่างเครื่องคอมพิวเตอร์ที่เป็นเซิร์ฟเวอร์ รวมถึง กฎระเบียบในการติดต่อด้วย

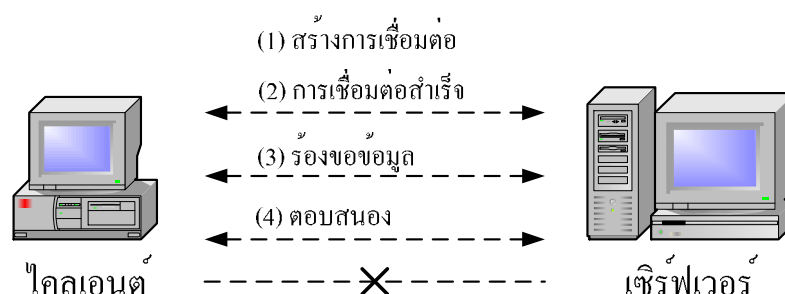
3.2 โพรโทคอลเอชทีทีพี (HTTP Hypertext Transfer Protocol)

โพรโทคอลเอชทีทีพี (HTTP Protocol) เป็นโพรโทคอลที่ใช้แลกเปลี่ยนข้อมูลกันระหว่างเซิร์ฟเวอร์และไคลเอนต์ของเวปไซต์ไวด์เว็บ โดยถูกออกแบบมาให้มีความกะทัดรัด สามารถทำงานได้รวดเร็ว มีกระบวนการทำงานที่ไม่ซับซ้อน และมีคำสั่งที่ใช้งานไม่มากนัก แต่สามารถรองรับข้อมูลได้ทุกแบบ ไม่ว่าจะเป็นข้อมูลทั่วไปที่เข้ารหัสแบบ เอ็มไอเอ็มอี (MIME) หรือข้อมูลที่เป็นกราฟิก เช่น ไฟล์ที่เป็นจีไอเอฟ (GIF) หรือ เจเป็ก (JPEG) เป็นต้น

หลักการทำงานทั่วไปของเอชทีทีพี (HTTP) จะแบ่งการทำงานออกเป็น 2 ด้าน คือ ด้านเว็บเซิร์ฟเวอร์และด้านไคลเอนต์ โดยไคลเอนต์จะติดต่อเข้ามายังเซิร์ฟเวอร์โดยใช้โปรแกรมบราวเซอร์และอ้างอิงแอดเดรสของเซิร์ฟเวอร์โดยใช้รูปแบบของยูอาแอล (URL) ส่วนด้านเซิร์ฟเวอร์จะส่งข้อมูลกลับมาในรูปแบบที่เป็นภาษาเอชทีเอ็มแอล (HTML - Hypertext Markup Language) โดยที่โพรโทคอลเอชทีทีพี (HTTP) ใช้วิธีการเข้ารหัสในแบบเอ็มไอเอ็มอี (MIME) เป็นมาตรฐานของการทำงาน

3.2.1 วิธีการติดต่อของโพรโทคอลเอชทีทีพี (HTTP Protocol)

การทำงานของโพรโทคอล (HTTP) เป็นแบบไคลเอนต์และเซิร์ฟเวอร์ (client/server) ดังนั้นการติดต่อสื่อสารใดๆ ผ่านโพรโทคอลนี้จำเป็นต้องมีเครื่องคอมพิวเตอร์ตัวลูกกับตัวแม่ การสื่อสารจึงจะสมบูรณ์ได้ ในการติดต่อกันระหว่างไคลเอนต์ไปยังเซิร์ฟเวอร์ผ่านโพรโทคอลเอชทีทีพี (HTTP) อย่างเช่น การเปิดเว็บเพจผ่านเว็บเบราว์เซอร์ จะมีขั้นตอน ดังนี้



รูปที่ 3.2 การติดต่อระหว่างไคลเอนต์กับเซิร์ฟเวอร์

จากรูปที่ 3.2 ขั้นแรกคือ ไคลเอนต์(ในตอนนี้คือเว็บเบราว์เซอร์) จะสร้าง การเชื่อมต่อ (Connection) กับเซิร์ฟเวอร์ผ่านสิ่งที่เรียกว่า “ซ็อกเก็ต” (socket) เมื่อซ็อกเก็ตทั้งสองฝั่งเชื่อมต่อกันได้สำเร็จ ไคลเอนต์จะส่งคำร้องขอข้อมูล (request) ไปยังเซิร์ฟเวอร์ จากนั้นเซิร์ฟเวอร์จะไปหาข้อมูลที่ไคลเอนต์ต้องการ ไม่ว่าจะหรือไม่มีข้อมูลตามที่ไคลเอนต์ร้องขอ เซิร์ฟเวอร์ก็จะต้องส่งข้อมูลตอบสนอง (response) กลับมายังไคลเอนต์เสมอ สุดท้ายการเชื่อมต่อจะถูกตัดขาดหรือปลดการเชื่อมต่อของซ็อกเก็ตทั้งสองฝั่งออก

การทำงานของโพรโทคอลเอชทีทีพี (HTTP) ที่มีการเชื่อมต่อในระยะเวลาเพียงสั้นๆ หรือที่เรียกว่าเป็นโพรโทคอลแบบ connectionless ทำให้ในช่วงเวลาหนึ่งๆ เซิร์ฟเวอร์ที่ให้บริการเวปไซต์ไวด์เว็บ (www)

สามารถรองรับไคลเอนต์ได้จำนวนมากพร้อมๆกัน เพราะไม่มีใครได้ทำการเชื่อมต่ออย่างถาวร

3.2.2 การจัดการข้อมูลของโปรโตคอลเอชทีทีพี

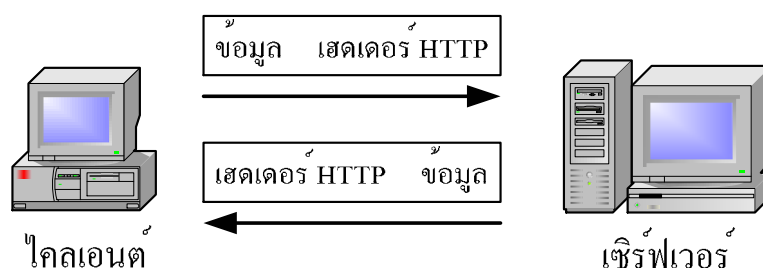
โปรโตคอลเอชทีทีพี (HTTP) ช่วยจัดการเรื่องข้อมูลที่ใช้ในการรับและส่งไฟล์ เพราะโปรโตคอลนี้จะระบุประเภทข้อมูล (data types) มากับข้อมูลเสมอ ทำให้หมดปัญหาเรื่องการตีความหมายของเนื้อหาไฟล์ไป ดังนั้นเราจึงสามารถส่งไฟล์ทุกประเภทผ่านทางโปรโตคอลนี้ ถ้าฝ่ายไคลเอนต์ได้รับไฟล์ไปแล้วทราบวิธีการแสดงผลของไฟล์นั้น ก็จัดการตามกรรมวิธีของไฟล์นั้นได้เลย แต่ถ้าหากไม่ทราบ ก็เป็นหน้าที่ของผู้ใช้เองที่ต้องหาโปรแกรมหรือแอปพลิเคชันมาจัดการกับไฟล์นั้นเอง

ตัวอย่างเช่น การเปิดไฟล์ผ่านเว็บเบราว์เซอร์ หากเป็นไฟล์นามสกุลคอตเอกซ์ทีเอ็ม(.htm), คอตเอกซ์ทีแอล (.html) หรือ คอตทีเอกซ์ที (.txt) แล้วเว็บเบราว์เซอร์จะนำข้อมูลแสดงออกทางวินโดว์ได้ทันที แต่ถ้าเป็นไฟล์นามสกุลพีดีเอฟ (.pdf) เว็บเบราว์เซอร์จะดูว่ามีโปรแกรมเสริมจำพวก ปลั๊กอิน (plug-in) สำหรับเปิดดูไฟล์ชนิดนี้หรือไม่ ถ้ามีก็จะเรียกปลั๊กอินให้มาช่วยนำข้อมูลไปแสดงผล ถ้าไม่มีปลั๊กอินหรือไม่เคยรู้จักนามสกุลนั้นมาก่อน ก็จะแสดงไอคอนล็อกบ็อกซ์ ถามความต้องการผู้ใช้อีกทีหนึ่งว่าจะให้จัดการต่อไปอย่างไร

3.2.3 โครงสร้างของโปรโตคอลเอชทีทีพี (HTTP)

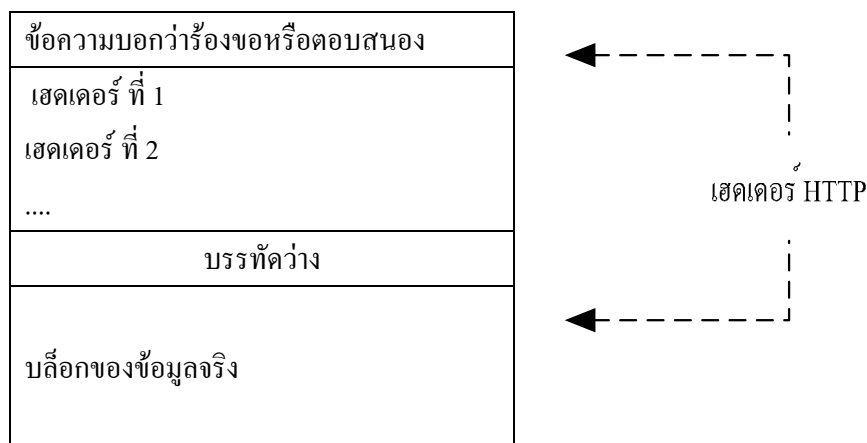
โครงสร้างข้อมูลของเอชทีทีพี (HTTP) จะแบ่งออกเป็น 2 ส่วนใหญ่ๆ คือ ส่วนเฮดเดอร์ หรือเรียกว่า metadata จะเป็นส่วนเก็บข้อมูลที่จำเป็นต้องใช้ภายในโปรโตคอล ส่วนที่สองเป็นส่วนข้อมูลจริงที่ต้องการรับส่ง ทั้งนี้เอชทีทีพี (HTTP) ถูกออกแบบมาให้สามารถรับส่งข้อมูลผ่านพร็อกซี (Proxy) หรือ ไฟร์วอลล์ (Firewall) ต่างๆได้ โดยการทำงานของเอชทีทีพี (HTTP) จะอาศัยโปรโตคอลพื้นฐานทีซีพี/ไอพี (TCP/IP) ซึ่งทั่วไปจะใช้หมายเลขพอร์ต 80

ในการร้องขอจากไคลเอนต์และการตอบสนองจากเซิร์ฟเวอร์ ย่อมต้องมีการรับส่งข้อมูลระหว่างกัน แต่ข้อมูลที่รับส่งให้กันในแต่ละครั้งไม่ได้มีเฉพาะข้อมูลเพียงอย่างเดียว แต่ละฝ่ายจะต้องเพิ่มส่วนที่เรียกว่า เฮดเดอร์เอชทีทีพี (HTTP Header) เข้าไปในส่วนหัวของข้อมูลด้วย เฮดเดอร์เอชทีทีพี จะใช้เป็นตัวบอกว่าข้อมูลที่ส่งเป็นข้อมูลการร้องขอจากไคลเอนต์หรือเป็นข้อมูลตอบสนองจากเซิร์ฟเวอร์ ดังรูปที่ 3.3



รูปที่ 3.3 ลักษณะข้อมูลที่รับส่งระหว่างไคลเอนต์กับเซิร์ฟเวอร์

เนื่องจากข้อมูลในเฮดเดอร์เฮชทีทีพี เป็นตัวควบคุมหรือบอกว่าให้ฝ่ายรับควรทำอะไรกับข้อมูลที่ส่งมาให้ ในบางครั้งจึงมีคนเรียกข้อมูลส่วนนี้ว่าข้อมูลเมต้า (Meta Information) ถึงแม้ว่าข้อมูลในเฮดเดอร์เฮชทีทีพี จะสำคัญต่อการนำไปตีความหมายของข้อมูลก็ตาม แต่ข้อมูลในเฮดเดอร์เฮชทีทีพี จะเป็นเพียงข้อความธรรมดา มิได้มีการเข้ารหัสหรือมีลักษณะพิเศษแตกต่างจากข้อมูลปกติแต่อย่างใด โดยโครงสร้างจะเป็นดังรูปที่ 3.4



รูปที่ 3.4 โครงสร้างของข้อมูลที่ส่งผ่านโพรโทคอลเฮชทีทีพี (HTTP Protocol)

3.2.4 ข้อความร้องขอ (request)

เมื่อไคลเอนต์เชื่อมต่อกับเซิร์ฟเวอร์สำเร็จแล้ว ไคลเอนต์ต้องเป็นฝ่ายเริ่มเปิดการพูดคุยก่อน ด้วยการส่งข้อมูลไปยังเซิร์ฟเวอร์เพื่อบอกการร้องขอข้อมูล การร้องขอไปยังเซิร์ฟเวอร์สามารถทำได้หลายแบบ เช่น ร้องขอให้เซิร์ฟเวอร์ส่งไฟล์มาให้แบบนี้จะเรียกว่าร้องขอแบบ GET, หรือร้องขอเพื่อถามเซิร์ฟเวอร์ว่ามีไฟล์ที่ต้องการอยู่ในเซิร์ฟเวอร์หรือไม่ (ถามเฉยๆไม่ต้องการให้เซิร์ฟเวอร์ส่งไฟล์จริงมาให้) แบบนี้จะเรียกว่าร้องขอแบบเฮด (HEAD), หากเป็นการร้องขอให้เซิร์ฟเวอร์รับข้อมูลจากไคลเอนต์ เรียกว่าร้องขอแบบโพสต์ (POST) ซึ่งการร้องขอแบบนี้หมายความว่า ไคลเอนต์ต้องการส่งข้อมูลไปให้เซิร์ฟเวอร์นั่นเอง แต่ในความเป็นจริงเราสามารถส่งข้อมูลไปยังเซิร์ฟเวอร์ด้วยการร้องขอแบบ GET ก็ได้

วิธีการร้องขอมียุคขึ้นอยู่กับเวอร์ชันของโพรโทคอลเฮชทีทีพี (HTTP) ที่ใช้ หากเป็นเวอร์ชัน 1.0 จะมีวิธีการร้องขอมาตรฐาน 3 วิธี คือ เก็ต (GET), เฮด (HEAD) และ โพสต์ (POST) แต่ถ้าใช้โพรโทคอลเฮชทีทีพี (HTTP) เวอร์ชัน 1.1 จะมีวิธีการร้องขอเพิ่มจากเวอร์ชัน 1.0 อีกหลายวิธี เช่น ออฟชั่น (OPTIONS), พุด (PUT), (DELETE) หรือ เทรค (TRACE) เป็นต้น

การใช้โพรโทคอลเฮชทีทีพี (HTTP) เวอร์ชันไหนนั้น ขึ้นอยู่กับเซิร์ฟเวอร์และไคลเอนต์ที่จะทำงานด้วย เช่น ถ้าทราบว่าเซิร์ฟเวอร์ที่จะติดต่อด้วยนั้น ใช้เว็บเซิร์ฟเวอร์ที่ทำงานสนับสนุนเฮชทีทีพี 1.1 ดังนั้นวิธีการร้องขอก็สามารถใช้ของเวอร์ชัน 1.1 ได้

ในส่วนของผู้ใช้ไคลเอนต์ โปรแกรมเว็บเบราว์เซอร์รุ่นใหม่ ๆ จะสนับสนุนเอชทีทีพีเวอร์ชัน 1.1 อยู่แล้ว ในกรณีที่เขียนซีจีไอ (CGI) เพื่อทำงานสร้างคำร้องไปยังเซิร์ฟเวอร์นั้นก็ไม่มีปัญหาใดๆ สามารถเขียนให้ ซีจีไอสร้างคำร้องขอแบบใดก็ได้ เพราะเป็นข้อความเพียงบรรทัดเดียว ดังนั้นปัญหาจึงขึ้นอยู่กับเซิร์ฟเวอร์ ที่จะติดต่อกับมากกว่า ว่าเซิร์ฟเวอร์จะรู้จักและสนับสนุนเอชทีทีพีเวอร์ชัน 1.1 หรือเวอร์ชันที่ใหม่กว่าได้หรือไม่

เพื่อไม่ให้เกิดปัญหาในการทำงาน ดังนั้นจึงควรใช้โพรโทคอลเอชทีทีพี (HTTP) เวอร์ชันที่เป็นกลางๆ อย่างเช่น เอชทีทีพี (HTTP) เวอร์ชัน 1.0 ซึ่งเว็บเซิร์ฟเวอร์ส่วนใหญ่จะใช้เวอร์ชันนี้กัน ในทางปฏิบัตินั้น เฉพาะวิธีร้องขอ 3 วิธีของ เอชทีทีพี (HTTP) เวอร์ชัน 1.0 ผสมกับการเขียนซีจีไอ ก็สามารถทำงานได้อย่างเพียงพอแล้ว

3.2.4.1 การร้องขอด้วยเมธอดเก็ท (GET)

ตัวอย่างในกรณีที่เว็บเบราว์เซอร์ขอเปิดเอกสารเอชทีเอ็มแอล (HTML) จากเซิร์ฟเวอร์จะร้องขอแบบเก็ท (GET) โดยเมื่อเว็บเบราว์เซอร์สร้างข้อเรียกเชื่อมต่อกับเซิร์ฟเวอร์ได้แล้วจะสร้างข้อความร้องขอขึ้นมาเพื่อส่งไปยังเซิร์ฟเวอร์ ข้อความในบรรทัดแรกของเฮดเดอร์เอชทีทีพี (HTTP Header) จะประกอบด้วย 3 ส่วน คือ วิธีการร้องขอ หรือที่เรียกว่า “เมธอด” (method), ไคลเอนต์กับชื่อไฟล์ที่ร้องขอ และเวอร์ชันของเอชทีทีพี (HTTP) ที่ผู้ร้องขอใช้อยู่แต่ละส่วนของข้อความจะถูกคั่นด้วยช่องว่าง (space) ฉะนั้นรูปแบบในการเขียนข้อความบรรทัดแรกของเฮดเดอร์เอชทีทีพี (HTTP) จึงเป็นดังนี้

GET /path/file HTTP /x.x

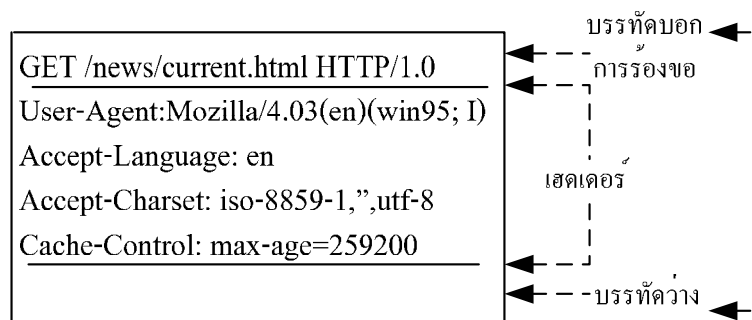
GET บอกว่าใช้เมธอดเก็ท (GET) ในการร้องขอข้อมูลจากเซิร์ฟเวอร์

/Path/file ไคลเอนต์และชื่อไฟล์ที่ต้องการจากเซิร์ฟเวอร์

HTTP/x.x.x.x คือ เวอร์ชันของเอชทีทีพี (HTTP) บอกแก่เซิร์ฟเวอร์ให้รู้ว่าทางไคลเอนต์ใช้เอชทีทีพี (HTTP) เวอร์ชันไหนอยู่

สังเกตว่าการขอข้อมูลจากเซิร์ฟเวอร์ จะระบุเฉพาะไคลเอนต์และชื่อไฟล์ที่ต้องการ ไม่มีการบอกชื่อโฮสต์และโดเมนเนมเพราะว่าเว็บเบราว์เซอร์ได้สร้างการเชื่อมต่อกับโฮสต์เอาไว้แล้ว การร้องขอข้อมูลจึงไม่ต้องระบุชื่อโฮสต์ซ้ำอีกครั้งนี้เราเรียกส่วนที่ระบุ /path/file นี้ว่า ยูอาร์ไอ (URI -Uniform Resource Identifier) ซึ่งคล้ายกับยูอาร์แอล (URL-Uniform Resource Locator) แต่สั้นกว่า เพราะไม่ต้องบอกโพรโทคอล, ชื่อโฮสต์ และโดเมนเนม แบบยาวเหยียด

ตัวอย่างเช่น ถ้าเราป้อนยูอาร์แอล (URL) ให้กับเว็บเบราว์เซอร์เป็น <http://ventura.lanna.com/news/current.html> เว็บเบราว์เซอร์ต้องสร้างการเชื่อมต่อไปยัง ventura.com ก่อนแล้วจึงร้องขอส่งไปยังเซิร์ฟเวอร์ สมมุติว่าเว็บเบราว์เซอร์ที่เราใช้สนับสนุนเอชทีทีพี (HTTP) เวอร์ชัน 1.0 ดังนั้นโครงสร้างของข้อความร้องขอจะเป็นดังรูปที่ 3.5

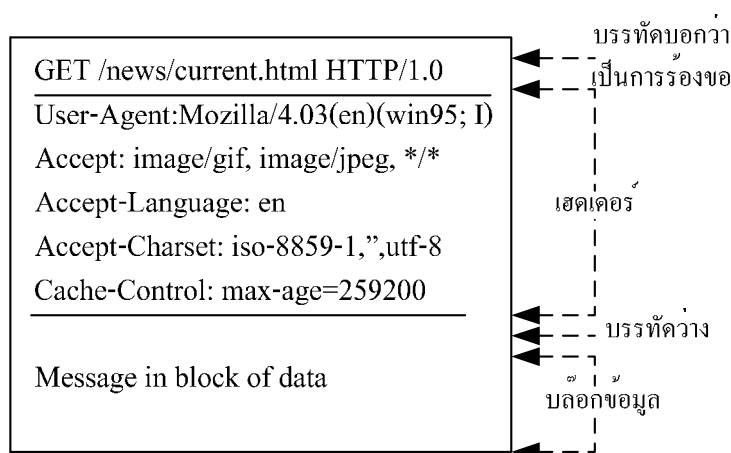


รูปที่ 3.5 ข้อความร้องขอที่ส่งไปยังเซิร์ฟเวอร์

จากรูปที่ 3.5 ตีความหมายจากแฮดเดอร์ไอชี่ทีพี (HTTP Header) ได้ว่า เว็บเบราว์เซอร์ร้องขอเปิดไฟล์ current.html ในไดเรกทอรี /news (จากโฮสต์เวนทูรา (ventura) ที่มีโดเมนเนมเป็น (lanna.com)) และเว็บเบราว์เซอร์ใช้โพรโทคอลไอชี่ทีพี (HTTP) เวอร์ชัน 1.0 จะสังเกตว่า นอกจากคำร้องขอที่อยู่ในบรรทัดแรกแล้ว ยังมีข้อมูลอื่นๆ ที่เว็บเบราว์เซอร์ส่งไปให้เซิร์ฟเวอร์ด้วย ข้อมูลในส่วนที่สองนี้เรียกว่า “แฮดเดอร์” (header) ข้อมูลในแฮดเดอร์ของเว็บเบราว์เซอร์แต่ละโปรแกรมแต่ละเวอร์ชันอาจไม่เหมือนกัน

จากโครงสร้างข้อมูลที่ได้รับระหว่างไคลเอนต์กับเซิร์ฟเวอร์ในรูปที่ 3.4 เราทราบว่าหลังจากแฮดเดอร์รายการสุดท้ายแล้ว จะมีบรรทัดว่างๆ แล้วตามด้วยบล็อกข้อมูล แต่เมื่อดูโครงสร้างของข้อความร้องขอที่ส่งไปยังเซิร์ฟเวอร์จากรูปที่ 3.5 หลังบรรทัดว่างๆ จะเห็นว่าไม่มีการบล็อกข้อมูลเลย ก็เพราะว่าในตอนนี้เป็นาร้องขอไฟล์ไปยังเซิร์ฟเวอร์ จึงไม่มีความจำเป็นที่ต้องส่งอะไรไปให้เซิร์ฟเวอร์มากกว่านี้ เพราะสิ่งที่ต้องการจากเซิร์ฟเวอร์ได้เขียนระบุไว้ในคำร้องขอบรรทัดแรกสุดแล้ว

ตัวอย่างในรูปที่ 3.6 เป็นการส่งข้อความร้องขอไปยังเซิร์ฟเวอร์ด้วยเมธอดเก้ด (GET) และมีการส่งข้อความว่า “Message in block of data” ไปให้เซิร์ฟเวอร์ด้วยถึงแม้จะทำแบบนี้ได้ แต่ไม่เกิดประโยชน์อะไร เพราะทางเซิร์ฟเวอร์ไม่ดึงเอาข้อความนั้นไปใช้งานแต่อย่างใด



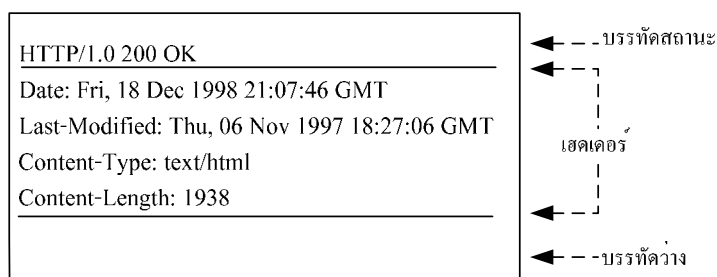
รูปที่ 3.6 ข้อความร้องขอที่มีข้อความส่งไปในบล็อกข้อมูลด้วย

3.2.4.2 การร้องขอด้วยเมธอดเฮด (HEAD)

การร้องขอด้วยเมธอดเฮด (HEAD) จะคล้ายกับเมธอดเก็ท (GET) คือใช้ร้องขอเพื่อถามเซิร์ฟเวอร์ และให้เซิร์ฟเวอร์ส่งรายละเอียดของไฟล์ที่ไคลเอนต์ต้องการกลับมา เพียงแต่เมธอดนี้จะไม่ส่งบล็อกข้อมูลไปกับข้อความร้องขอเหมือนกับเมธอดเก็ท (GET)

เมธอดนี้มีประโยชน์สำหรับการตรวจสอบว่า มีไฟล์ที่ต้องการอยู่ในเซิร์ฟเวอร์หรือไม่ หรือใช้ตรวจสอบความสมบูรณ์ของลิงก์ก็ได้ รหัสตอบสนองในบรรทัดสถานะจึงอาจเป็น 200 หรือ 404 และอาจมีข้อมูลอื่นส่งเพิ่มเติมมาในเฮดเดอร์ด้วย เช่น วันที่ปรับปรุงแก้ไขไฟล์ครั้งสุดท้าย (Last Modified) เป็นต้น

สมมุติว่ามีการส่งข้อความร้องขอโดยใช้เมธอดเก็ท (GET) ไปยังเซิร์ฟเวอร์ `ventura.lanna.com` เพื่อถามว่ามีไฟล์ `index.html` หรือไม่ โดยระบุข้อความร้องขอในบรรทัดแรกเป็น `HEAD/index.html` เอชทีทีพี (HTTP)/1.0 และสมมุติว่ามีไฟล์ `index.html` อยู่ในเซิร์ฟเวอร์จริง และไฟล์นี้มีขนาด 1,938 ไบต์ ข้อความตอบกลับจากเซิร์ฟเวอร์ก็จะได้อ้างรูปที่ 3.7



รูปที่ 3.7 ข้อความตอบสนองที่ได้จากการร้องขอด้วยเมธอดเฮด (HEAD)

ในการร้องขอแบบเฮด (HEAD) นั้นข้อความตอบกลับจากการร้องขอจะไม่มีการส่งเนื้อหาของไฟล์มาให้ (ไม่มีการบล็อกข้อมูล) ถึงแม้ว่าไฟล์ที่ร้องขอไปจะมีอยู่จริงในเซิร์ฟเวอร์ก็ตาม การสอบถามข้อมูลของไฟล์ในเซิร์ฟเวอร์ด้วยเมธอดเฮด (HEAD) จึงทำงานได้คำตอบเร็วกว่าใช้เมธอดเก็ท (GET) ตัวอย่างการใช้งานเมธอดเฮด (HEAD) คือ มีบางโปรแกรมทำงานโดยให้ผู้กำหนดชื่อไฟล์ที่ต้องการจากเซิร์ฟเวอร์ หลังจากนั้นโปรแกรมจะคอยวิ่งไปเช็คกับทางเซิร์ฟเวอร์เป็นช่วงๆ ตามระยะเวลาที่ตั้งไว้ (โปรแกรมร้องขอไปยังเซิร์ฟเวอร์ด้วยเมธอดเฮด) หากพบว่าไฟล์ในเซิร์ฟเวอร์มีการปรับปรุง คือ วันที่ของไฟล์ในเซิร์ฟเวอร์ใหม่กว่าไฟล์ที่เก็บอยู่ในเครื่อง โปรแกรมก็จะดาวน์โหลดไฟล์นั้นมาทับไฟล์เดิมในเครื่อง (ร้องขอไฟล์นั้นอีกครั้งด้วยเมธอดเก็ท) เป็นต้น

3.2.4.3 การร้องขอด้วยเมธอดโพส (POST)

สำหรับการร้องขอด้วยเมธอดโพส (POST) จะใช้ในกรณีที่ต้องการส่งข้อมูลจากไคลเอนต์ไปยังเซิร์ฟเวอร์ คือ ไม่ได้ขอไฟล์จากเซิร์ฟเวอร์ แต่จะส่งข้อมูลไปให้เซิร์ฟเวอร์รับไปทำงาน เมื่อเซิร์ฟเวอร์รับข้อความร้องขอแบบโพส (POST) แล้ว ก็จะทราบว่าไคลเอนต์ต้องการส่งข้อมูลมาให้ รูปแบบข้อความร้องขอในบรรทัดแรกจะเขียนคล้ายกับเมธอดเก็ท (GET) ดังนี้

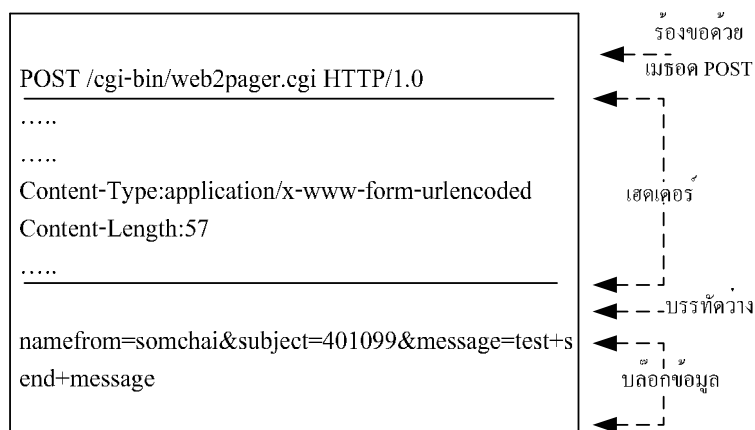
การร้องขอด้วยเมธอด POST จะมีเงื่อนไขต่อไปนี้

- ข้อมูลที่จะส่งไปให้เซิร์ฟเวอร์จะอยู่ภายในบล็อกข้อมูล ดังนั้นจึงต้องมีเฮดเดอร์เพื่อบอกรายละเอียดของข้อมูลในบล็อกข้อมูลแนบไปด้วย เช่น ข้อมูลเป็นแบบไหน (Content-Type:) และข้อมูลมีกี่ไบต์(Content-Length:) เสมอ
- /path/file คือชื่อโปรแกรมซีจีไอ (CGI) ในเซิร์ฟเวอร์ที่จะทำหน้าที่รับข้อมูลไปประมวลผล
- ข้อความตอบสนองที่เซิร์ฟเวอร์จะส่งกลับให้ไคลเอนต์ จะได้มาจากการทำงานของโปรแกรม หรือซีจีไอ (CGI) ในเซิร์ฟเวอร์ ดังนั้นซีจีไอ (CGI) ที่รับข้อมูลไปจึงต้องทำหน้าที่ส่งข้อความตอบกลับไปให้ไคลเอนต์

โดยส่วนใหญ่การส่งข้อมูลจากฟอร์มในเว็บเพจไปประมวลผลด้วยซีจีไอ (CGI) ในเซิร์ฟเวอร์ จะใช้เมธอดนี้มากที่สุด ความจริงแล้วการส่งข้อมูลไปยังเซิร์ฟเวอร์สามารถใช้เมธอดเก็ท (GET) ก็ได้ แต่มีข้อจำกัดเรื่องความยาวของข้อมูลที่จะส่งไปให้เซิร์ฟเวอร์ ถ้าใช้เมธอดโพส (POST) เพื่อร้องขอส่งข้อมูลไปยังเซิร์ฟเวอร์ เฮดเดอร์ที่ชื่อ Content-Type จะถูกกำหนดให้เป็น application/x-www-form-urlencoded เพื่อบอกแก่เซิร์ฟเวอร์ว่าข้อมูลที่ส่งไปให้มีการเข้ารหัสและเฮดเดอร์ Content-Length จะใช้สำหรับบอกความยาวของข้อมูลที่เข้ารหัสแล้ว เพราะข้อมูลที่กรอกผ่านอินพุตในฟอร์มจะถูกเว็บเบราว์เซอร์เข้ารหัสก่อนส่งเสมอ การเข้ารหัสนี้เรียกว่า ยูอาแอลเอ็นโค้ด (URL-encode) ซึ่งมีรายละเอียด ดังนี้

- แปลงตัวอักษรหรือเครื่องหมายบางตัวให้อยู่ในรูป %xx โดยที่ xx จะเป็นค่ารหัสแอสกีของอักขระตัวนั้น ตัวอักษรที่ต้องมีการแปลง เช่น =, &, % และ + เพราะเป็นเครื่องหมายที่ใช้เป็นตัวแบ่งแยกข้อมูลที่จะส่งไปยังเซิร์ฟเวอร์
- เปลี่ยนช่องว่าง (space) ทุกตัวเป็นเครื่องหมายบวก (+)
- รวมชื่อตัวแปรและค่าตัวแปรเข้าด้วยกัน โดยคั่นกลางด้วยเครื่องหมาย = และคั่นกลางระหว่างตัวแปรด้วยเครื่องหมาย & เช่น number=152-401099&from=zorkia&message= Just + a + test

ตัวอย่างข้อความร้องขอด้วยเมธอด POST ที่เว็บเบราว์เซอร์สร้างขึ้นเพื่อส่งข้อมูลจากฟอร์มในเว็บฟอร์มในเว็บเพจไปให้แกซีจีไอ (CGI) ที่ชื่อ web2pager.cgi ในไดเรกทอรี /cgi-bin ของเซิร์ฟเวอร์ <http://203.146.1.214> รับไปทำงานเป็นดังรูปที่ 3-8



รูปที่ 3.8 ตัวอย่างข้อความร้องขอด้วยเมธอดโพส (POST)

3.2.5 ข้อความตอบสนอง (response)

ไม่ว่าจะมีข้อมูลหรือไม่มีข้อมูลที่ไคลเอนต์ร้องขอเข้ามาที่ตามเซิร์ฟเวอร์จะต้องส่งข้อความตอบสนองกลับไปให้ไคลเอนต์รับทราบเสมอ ในกรณีของการร้องขอข้อความบรรทัดแรกสุดจะบ่งบอกวิธีการร้องขอ แต่ถ้าเป็นกรณีตอบสนอง ข้อความบรรทัดนี้จะเรียกว่าเป็น “บรรทัดสถานะ” (status line) โครงสร้างในบรรทัดนี้มี 3 ส่วน แต่ละส่วนคั่นด้วยช่องว่าง ดังต่อไปนี้

HTTP/ x. x xxx Description

- HTTP/ x. x x. x คือ เวอร์ชันของ HTTP
 - xxx ตัวเลข 3 หลัก เป็นรหัสตอบสนอง (response status code)
 - Description ข้อความอธิบายรหัสตอบสนอง ให้อ่านเข้าใจได้ง่ายขึ้น
- ตัวเลขหลักแรกของรหัสตอบสนอง เป็นรหัสบอกหมวดหมู่ของการตอบสนอง ดังต่อไปนี้
- 1xx ข้อความข่าวสารอย่างเดียว
 - 2xx การร้องขอสำเร็จ
 - 3xx เปลี่ยนทิศทางของไคลเอนต์ไปยัง URL อื่น
 - 4xx มีความผิดพลาดที่เกิดจากการร้องขอ
 - 5xx มีความผิดพลาดทางด้านเซิร์ฟเวอร์รหัสตอบสนองที่ต้องเจอบ่อยๆ

เช่น...

รหัส 200 หมายถึง ร้องขอสำเร็จ คือ มีไฟล์ที่ต้องการอยู่ในเซิร์ฟเวอร์และเซิร์ฟเวอร์ได้ส่งเนื้อหาของไฟล์มาให้ไคลเอนต์ด้วยแล้ว

รหัส 404 หมายถึง ไม่มีไฟล์ที่ต้องการอยู่ในเซิร์ฟเวอร์

รหัส 500 หมายถึง มีความผิดพลาดในการทำงานของเซิร์ฟเวอร์

... เป็นต้น

ตัวอย่างเช่น เราป้อน URL ให้กับเว็บเบราว์เซอร์เป็น <http://ventura.lanna.com/news/current.html> ซึ่งเป็นการขอเปิดไฟล์ current.html ในไดเรกทอรี /news จากเซิร์ฟเวอร์ ventura.lanna.com สมมุติว่ามีไฟล์นี้ในเซิร์ฟเวอร์จริง ขนาดไฟล์เป็น 96 ไบต์ และเนื้อหาในไฟล์ current.html เป็นดังต่อไปนี้

```
<html>

<head><title>Lanna</title></head>

<body>

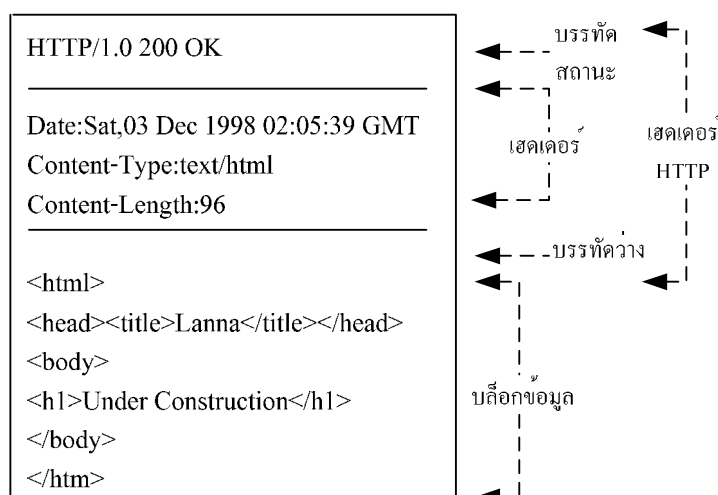
<h1>Under Construction </h1>

</body>

</html>
```

เมื่อเว็บเบราว์เซอร์ได้รับคำร้องขอแล้ว ไปตรวจสอบพบว่าไม่มีไฟล์ที่เว็บเบราว์เซอร์ต้องการอยู่จริง ดังนั้นเว็บเบราว์เซอร์จะส่งข้อความตอบสนองกลับ โดยที่ข้อความในบรรทัดสถานะจะเป็น....

HTTP/1.0 200 OK



รูปที่ 3.9 ข้อความตอบสนองที่ส่งมาจากเซิร์ฟเวอร์

และเนื้อหาของไฟล์ current.html นั้นจะถูกส่งมาให้เว็บเบราว์เซอร์ โดยจะอยู่ในบล็อกข้อมูลเพื่อให้เว็บเบราว์เซอร์นำไปตีความหมายและแสดงในวินโดว์เว็บเบราว์เซอร์ ถ้าจำลองตัวเองเป็นเว็บเบราว์เซอร์หลังจากที่ส่งข้อความร้องขอไปยังเซิร์ฟเวอร์แล้ว อาจจะเห็นข้อมูลในลักษณะดังรูปที่ 3.9 ตอบกลับมาจากเซิร์ฟเวอร์

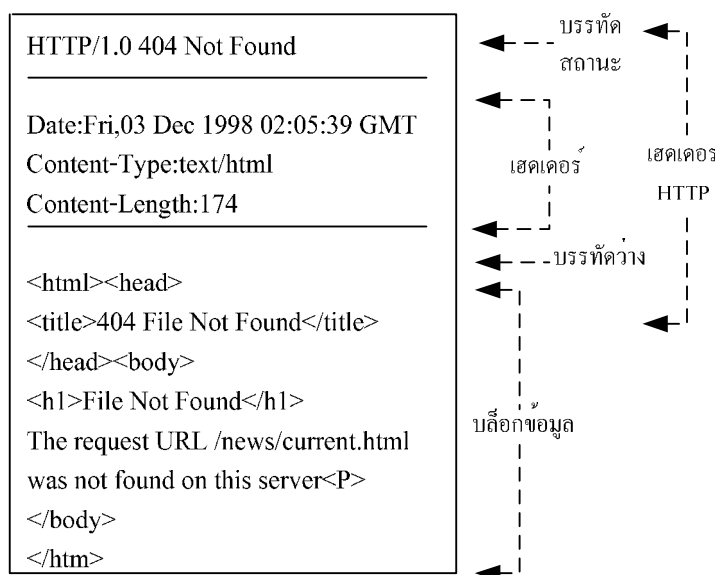
จากรูปที่ 3.9 จะเห็นว่าในส่วนของเฮดเดอร์ จะมีรายการเฮดเดอร์อยู่ 3 บรรทัด บรรทัดแรกที่มีคำว่า Date: เป็นตัวบอกวัน เดือน ปี ที่ข้อความตอบสนองถูกสร้างขึ้น บรรทัดที่ 2 Content-type: บอกข้อมูลที่อยู่

ที่ส่งมาอยู่ในบล็อกข้อมูลมีเนื้อหาแบบ text/html และบรรทัดสุดท้าย Content-Length: บอกจำนวนไบต์ของข้อมูลที่ส่งมา (จำนวนไบต์ของข้อมูลในบล็อกข้อมูล)

แต่หากไม่มีไฟล์ current.html อยู่ที่เซิร์ฟเวอร์ ดังนั้นเซิร์ฟเวอร์ต้องตอบกลับให้เว็บเบราว์เซอร์ทราบว่าไม่มีไฟล์ดังกล่าว โดยข้อความในบรรทัดสถานะจากเหตุการณ์นี้จะเป็น...

HTTP/1.0 404 Not Found

ตัวเลขรหัส 404 คือ รหัสบอกว่ามีความผิดพลาดในการร้องขอ และคำว่าค้นหา Not Found เป็นคำอธิบายความหมายของรหัส 404 ฉะนั้นถ้าจำลองตัวเราเป็นเว็บเบราว์เซอร์อีกครั้ง ก็อาจเห็นข้อมูลลักษณะดังรูปที่ 3.10 ตอบกลับมาจากเซิร์ฟเวอร์



รูปที่ 3.10 ข้อความตอบสนองจากเซิร์ฟเวอร์เมื่อไม่มีข้อมูลที่ร้องขอ

3.2.6 ข้อแตกต่างระหว่างเวอร์ชัน 1.0 กับ เวอร์ชัน 1.1

โปรโตคอลเวอร์ชันใหม่ในเอชทีทีพี (HTTP) เวอร์ชัน 1.1 นี้ได้เพิ่มประสิทธิภาพทำงานให้สูงขึ้น และปรับปรุงในด้านต่างๆ ที่ทำให้มีความสามารถมากขึ้น ดังนี้

- ลดภาระของการเชื่อมต่อผ่านโปรโตคอลทีซีพี (TCP Protocol) และสามารถใช้ประสิทธิภาพของทีซีพี (TCP) ได้อย่างเต็มที่
- สามารถทำการบีบอัดข้อมูลที่รับส่งระหว่างเซิร์ฟเวอร์และไคลเอนต์ได้
- รองรับการทำงานแบบ virtual host หมายถึง เว็บเซิร์ฟเวอร์เครื่องหนึ่งๆ มีชื่อโดเมนมากกว่าหนึ่งชื่อได้
- สามารถรองรับการทำงานได้หลายภาษา

- โอนไฟล์ข้อมูลเฉพาะบางส่วนได้ คุณสมบัติมีประโยชน์มากในกรณีที่มีการโอนไฟล์ข้อมูลขนาดใหญ่และเกิดปัญหาขึ้นระหว่างการทำงาน ซึ่งโปรโตคอลเอชทีทีพี (HTTP Protocol) เวอร์ชัน 1.1 มีจุดเด่นที่สามารถตรวจสอบได้ และโอนไฟล์ข้อมูลต่อจากส่วนที่เคยโอนมาแล้วได้

เอชทีทีพี (HTTP) มีคำสั่งต่างๆ ไม่มากนัก รายละเอียดคำสั่งของเอชทีทีพี (HTTP) มีดังนี้

คำสั่ง	รายละเอียด
GET	ใช้อ่านข้อมูลจากเว็บเซิร์ฟเวอร์และส่งไปยังไคลเอนต์ โดยมีรูปแบบดังนี้ GET<URL> HTTP/1.0 ตัวอย่างเช่น ต้องการให้เว็บเซิร์ฟเวอร์ส่งไฟล์ sale.html จากโดเมน www.netcorp.com ไปยังไคลเอนต์จะใช้รูปแบบของคำสั่ง GET ดังนี้ GET www.netcorp.com/sale.html/HTTP/1.0 นอกจากนี้คำสั่ง GET ยังสามารถกำหนดเงื่อนไขให้อ่านข้อมูลจากเว็บเซิร์ฟเวอร์ เฉพาะที่มีการเปลี่ยนแปลงแก้ไขได้ด้วย
HEAD	คำสั่งนี้จะทำงานคล้ายกับคำสั่ง GET แต่เว็บเซิร์ฟเวอร์จะส่งข้อมูลกลับมาให้เฉพาะในรายละเอียดของ metadata หรือข้อมูลในเฮดเดอร์เท่านั้น ส่วนข้อมูลที่เป็น HTML จะไม่ถูกส่งมาด้วย ซึ่งคำสั่ง HEAD นี้จะใช้เพื่อทดสอบว่าข้อมูลตาม URL นั้นๆมีการเปลี่ยนแปลงหรือไม่เท่านั้น
POST	เป็นคำสั่งที่ตรงข้ามกับคำสั่ง GET และ HEAD โดยทำหน้าที่ส่งข้อมูลจากไคลเอนต์ไปยังเว็บเซิร์ฟเวอร์ แต่โดยปกติแล้วการส่งข้อมูลจากไคลเอนต์ไปยังเว็บเซิร์ฟเวอร์นั้นจะไม่ค่อยมีใช้งาน นอกจากในกรณีที่ HTML ทำงานในลักษณะที่ให้ผู้ใช้อัปโหลดข้อมูลลงตามแบบฟอร์ม (เช่น รายละเอียดส่วนตัวของผู้ใช้งาน) และส่งข้อมูลนี้กลับมาเก็บไว้ที่เว็บเซิร์ฟเวอร์
PUT	เป็นคำสั่งที่ทำงานเหมือนกับคำสั่ง POST แต่ไม่เป็นที่นิยม
DELETE	เพื่อให้ไคลเอนต์สั่งเว็บเซิร์ฟเวอร์ลบ URL ที่กำหนดไว้ออกจากเซิร์ฟเวอร์ แต่เป็นคำสั่งที่ไม่นิยมใช้มากนัก เนื่องจากเว็บเซิร์ฟเวอร์ทั่วไปมักจะทำงานในแบบอ่านข้อมูลได้เท่านั้น (read-only)
LINK	เป็นคำสั่งที่เชื่อม URL ที่ต้องการไปยังเว็บเซิร์ฟเวอร์อื่น
UNLINK	ยกเลิกคำสั่ง LINK ให้กลับมาใช้เซิร์ฟเวอร์เดิมตามที่กำหนดไว้ใน URL

ตารางที่ 3.1 แสดงคำสั่งของเอชทีทีพี (HTTP)

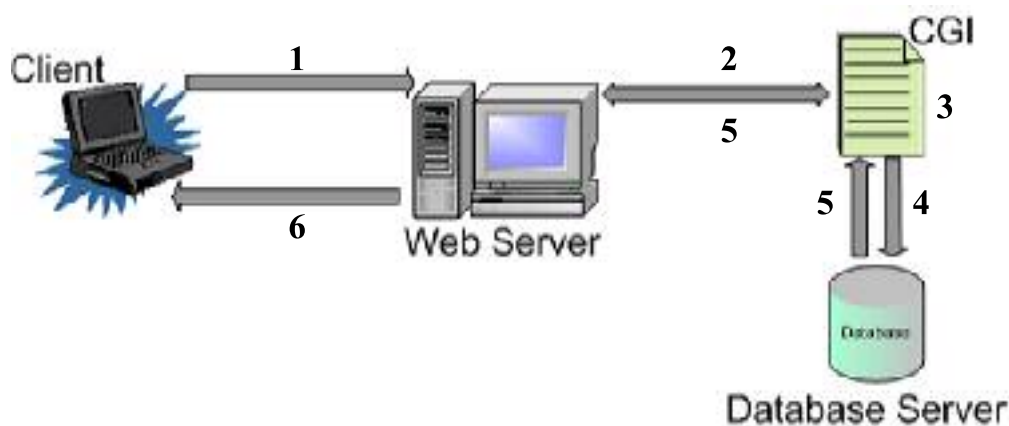
3.3 ซีจีไอ (CGI Common Gateway Interface)

การทำงานของเวิลด์ไวด์เว็บ (www) แต่เดิมเป็นการแสดงข้อมูลที่คงที่ที่ไม่มีการเปลี่ยนแปลงและไม่สามารถตอบโต้กับผู้ใช้งานได้ แต่ซีจีไอ (CGI) ซึ่งย่อมาจาก Common Gateway Interface เป็นกลไกที่ทำให้

ให้เครื่องผู้ให้บริการสามารถเรียกโปรแกรมประยุกต์บนเครื่องให้บริการได้ การแสดงผลของเว็บจึงไม่ได้เป็นแบบคงที่อีกต่อไปเพราะมันเรียกใช้งานโปรแกรมประยุกต์บนเครื่องให้บริการได้ การแสดงผลของเว็บจึงไม่ได้เป็นแบบคงที่อีกต่อไปเพราะมันไม่ได้เก็บข้อมูลที่ไฟล์ใด แต่ผลลัพธ์ที่เกิดขึ้นมาจากการรันแอปพลิเคชันทางด้านเครื่องให้บริการแล้วส่งข้อมูลที่ได้ออกไปให้ผู้ใช้งานฝั่งบราวเซอร์ทันที โดยธรรมชาติข้อมูลบางอย่างมันมีการเปลี่ยนแปลงอยู่เสมอ เช่น ข้อมูลการซื้อขายหุ้นในตลาดหลักทรัพย์ เป็นต้น ดังนั้นจึงเป็นการดีที่จะใช้โปรแกรมซีจีไอ (CGI) ดึงข้อมูลล่าสุดมาเก็บไว้ในฐานข้อมูลเพื่อที่จะนำไปแสดงผลต่อไปยังบราวเซอร์ อีกตัวอย่างหนึ่งของการใช้ซีจีไอ (CGI) คือ การอนุญาตให้ผู้ที่ไม่ใช่สมาชิกเท่านั้นจะสามารถที่จะเข้าถึงข้อมูลที่ต้องการได้ ในขณะที่ผู้ใช้สมาชิกจะสามารถเข้าดูข้อมูลได้เฉพาะที่อยู่ภายนอกเท่านั้น ในปัจจุบันการดำเนินธุรกิจการค้าทางอินเทอร์เน็ตได้เพิ่มมากขึ้นที่เรียกว่า อีคอมเมิร์ซ (E-Commerce) ก็ได้นำความสามารถทางด้านซีจีไอ (CGI) มาใช้อย่างมาก คือ สามารถที่จะรับการสั่งซื้อจากลูกค้าผ่านทางอินเทอร์เน็ต และก็ให้ลูกค้าสามารถจ่ายเงินผ่านทางอินเทอร์เน็ตได้ด้วย

3.3.1 หลักการทำงานของซีจีไอ (CGI)

ซีจีไอ (CGI) เป็นแอปพลิเคชัน ซึ่งทำงานเป็นตัวกลางระหว่างเซิร์ฟเวอร์กับไคลเอนต์ โดยข้อมูลที่จะให้บริการผ่านเว็บไซต์เว็บ (www) ได้นั้น จะต้องถูกเก็บอยู่ในเซิร์ฟเวอร์ หรือ ฐานข้อมูล เมื่อทางฝั่งเซิร์ฟเวอร์ได้รับการร้องขอไฟล์จากไคลเอนต์ ทางฝั่งเซิร์ฟเวอร์จะค้นหาและส่งไฟล์ที่ไคลเอนต์ต้องการกลับไปให้ไคลเอนต์ แสดงดังรูปที่ 3.11



รูปที่ 3.11 แสดงการทำงานของซีจีไอ (CGI)

จากรูปที่ 3.11 จะมีขั้นตอนการทำงานเป็นขั้นตอนต่างๆ ดังนี้

ขั้นตอนที่ 1 ฝั่งไคลเอนต์ (Client) จะทำการร้องขอข้อมูล หรือ เรียกใช้ไฟล์ PHP ที่เก็บอยู่ในเครื่องเซิร์ฟเวอร์ (Server)

ขั้นตอนที่ 2 ฝั่งเซิร์ฟเวอร์จะทำการค้นหาไฟล์ PHP แล้วทำการประมวลผลไฟล์ PHP ตามที่ไคลเอนต์ทำการร้องขอมา

ขั้นตอนที่ 3 ทำการประมวลผลไฟล์ PHP

ขั้นตอนที่ 4 ทำการติดต่อกับฐานข้อมูลโดยส่งคำสั่งเอสคิวแอล (SQL) ไปยังคิบีเอ็มเอส (DBMS) เพื่อขอข้อมูลในฐานข้อมูลมาใช้ในการประมวลผล

ขั้นตอนที่ 5 DBMS จะทำการประมวลผลคำสั่งเอสคิวแอล (SQL) ที่ได้แล้วส่งค่าของข้อมูลที่ต้องการกลับไปยังเซิร์ฟเวอร์

ขั้นตอนที่ 6 ส่งผลลัพธ์จากการประมวลผลไปยังเครื่องไคลเอนต์

ลักษณะการทำงานของซีจีไอ (CGI) ต้องอาศัยการประมวลผลที่เซิร์ฟเวอร์ แล้วสร้างคำตอบออกมาเป็นเนื้อหาแบบเอชทีเอ็มแอล (HTML) จากนั้นจึงส่งเนื้อหากลับไปให้ไคลเอนต์ ซึ่งเป็นการทำงานแบบรวมศูนย์ คือ งานทุกอย่างทำที่เซิร์ฟเวอร์หมด ไคลเอนต์เพียงแต่ทำหน้าที่ส่งคำร้องขอและรอรับผลการทำงานเท่านั้น ดังนั้นเซิร์ฟเวอร์จะต้องทำงานหนักมากและใช้เนื้อที่ในการเก็บข้อมูลเยอะแต่เพื่อเป็นการลดภาระให้แก่เซิร์ฟเวอร์เราสามารถที่จะใช้ Cookie ช่วยได้โดยให้ทางฝั่งไคลเอนต์ช่วยเก็บข้อมูลของผู้ใช้แต่ละคนไว้แทน

3.3.2 ภาษาที่ใช้ในการเขียนโปรแกรมซีจีไอ (CGI)

เนื่องจากซีจีไอ เป็นโปรแกรมการอินเทอร์เน็ตเฟสเบื้องต้น ดังนั้น จึงใช้ภาษาอะไรก็ได้ที่สามารถทำงานได้บนระบบปฏิบัติการเดียวกับเว็บเซิร์ฟเวอร์ ไม่ว่าจะเป็น PERL, C/C++, Visual Basic, Delphi, Pascal และอื่นๆ แต่การจะเลือกว่าควรจะใช้ภาษาใดนั้นขึ้นอยู่กับตัวแปรหลายอย่างเพราะแต่ละภาษามีจุดเด่นที่แตกต่างกัน เช่น

PERL เป็นภาษาแบบสคริปต์ที่ทำงานบนระบบปฏิบัติการยูนิกซ์ แต่ปัจจุบันได้พัฒนาให้สามารถทำงานบนระบบปฏิบัติการวินโดวส์ได้ด้วย ใช้งานง่ายโดยเฉพาะมีความสามารถในการจัดการกับตัวอักษรได้เป็นอย่างดี จึงเป็นที่นิยมในการทำโปรแกรมซีจีไอ (CGI)

Visual Basic เป็นภาษาที่ได้รับความนิยมมากสำหรับการเขียนโปรแกรมบนระบบปฏิบัติการวินโดวส์ เนื่องจาก Visual Basic เป็นภาษาที่ได้รับความนิยมในการเขียนโปรแกรมต่างๆทั่วโลก และอีกเหตุผลหนึ่งเพราะว่า ภาษา Visual Basic มีความง่ายในการศึกษาและการใช้งาน

ภาษาซี (C) ก็เป็นอีกภาษาหนึ่งที่เหมาะสำหรับใช้เขียนโปรแกรมซีจีไอ (CGI) เหตุผลเช่นเดียวกันกับภาษา Visual Basic

พีเอชพี (PHP) เป็นสคริปต์แบบเอ็มเบ็ดดัด Embedded คือ สามารถแทรกร่วมกับแท็กเอชทีเอ็มแอล (HTML Tag) ได้อย่างอิสระ และหากเราพัฒนาโค้ดไว้ในรูปแบบของชั้น (Class) ที่เขียนขึ้นเพียงครั้งเดียวแล้วเรียกใช้งานได้ตลอดทำให้สะดวกและรวดเร็วต่อการพัฒนาโปรแกรมต่างๆ

ดังนั้นการเลือกว่าควรใช้ภาษาใดนั้นขึ้นอยู่กับงานที่ซีจีไอ (CGI) จะต้องทำและความถนัดในภาษานั้นๆ สิ่งแรกที่สำคัญในการเลือกใช้ คือ ภาษาที่เลือกใช้สามารถรันได้บนเครื่องเซิร์ฟเวอร์ ไหนได้บ้าง